

Modul Praktikum Flutter Lanjutan

Pertemuan 12: Dashboard Admin dengan Laravel dan Filament

Durasi	Level	Bentuk kerja	Prasyarat
2 x 150 menit	Lanjut	Individu	Pertemuan 11 (aplikasi memakai data server)

1. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Menormalkan data master: memindahkan kategori dari daftar tetap di kode ke tabel `kategori`, termasuk migrasi data lama.
2. Menambahkan layanan Laravel + Filament ke Docker Compose yang **berbagi database** dengan FastAPI.
3. Menetapkan kepemilikan skema: Alembic untuk tabel domain, migrasi Laravel hanya untuk tabel milik Laravel.
4. Membuat Filament Resource untuk CRUD, daftar baca-saja, filter, dan saklar (*toggle*).
5. Menunjukkan alur ujung ke ujung: perubahan admin di dashboard langsung terlihat di aplikasi mobile.

2. Alat dan Bahan

- Hasil pertemuan 11 (atau snapshot `pert11/tugas11`).
- Image `dunglas/frankenphp:1-php8.4` dan `composer:2` (sekitar 600 MB). **Unduh sebelum kelas.**
- Tidak perlu memasang PHP di laptop: semua perintah `php artisan` dijalankan di dalam container.
- Port baru: **8080** (dashboard admin).

3. Teori Singkat

Mengapa Laravel + Filament? Membuat dashboard admin dari nol (login, tabel, form, filter, grafik) memakan waktu berminggu-minggu. **Filament** adalah kerangka panel admin untuk Laravel yang membuat halaman CRUD dari deklarasi singkat (*Resource*). FastAPI tetap menjadi API untuk aplikasi mobile, sedangkan Filament khusus untuk manusia (admin) di browser.

Satu database, dua aplikasi. Kedua aplikasi membaca dan menulis tabel yang sama. Aturan agar skema tidak bentrok:

Tabel	Pemilik skema	Keterangan
<code>users, kegiatan, kategori, alembic_version</code>	Alembic (FastAPI)	Laravel hanya memetakan dengan model Eloquent, tanpa migrasi Laravel
<code>admins, sessions, cache, jobs, migrations</code>	Migrasi Laravel	Kebutuhan internal Laravel/Filament

****Bentrok nama tabel `users`.**** Laravel secara bawaan memakai tabel `users` untuk login, padahal `users` di sini berisi pengguna aplikasi mobile yang password-nya di-hash oleh FastAPI. Karena itu login dashboard dipindah ke tabel ****admins**. **Admin juga tidak membuat akun pengguna mobile; admin hanya dapat menonaktifkan**** akun (kolom `aktif` dari pertemuan 10).

Normalisasi kategori. Kategori yang tadinya daftar tetap di Flutter dan Python dipindah ke tabel kategori, sehingga admin dapat menambah, mengurutkan, atau menonaktifkan kategori tanpa menulis ulang aplikasi. Kolom teks kegiatan.kategori diganti dengan *foreign key* kegiatan.kategori_id. FK dibuat ON DELETE RESTRICT: kategori yang sudah dipakai tidak dapat dihapus, cukup dinonaktifkan.

4. Langkah Praktikum (Sesi 1: Backend)

Bagian A: Tabel Kategori dan Migrasi Data

Tambahkan model Kategori di models.py, lalu ganti kolom kategori pada Kegiatan:

```
class Kategori(Base):
    """Kategori pengeluaran. Dikelola admin lewat dashboard Filament."""

    __tablename__ = "kategori"

    id: Mapped[int] = mapped_column(primary_key=True)
    nama: Mapped[str] = mapped_column(String(30), unique=True)
    urutan: Mapped[int] = mapped_column(default=0)
    aktif: Mapped[bool] = mapped_column(default=True)
    created_at: Mapped[datetime] = mapped_column(DateTime, server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, server_default=func.now(), onupdate=func.now()
    )

class Kegiatan(Base):
    ...
    kategori_id: Mapped[int | None] = mapped_column(
        ForeignKey("kategori.id", ondelete="RESTRICT"), index=True
    )
    ...
    kategori: Mapped[Kategori | None] = relationship()

    @property
    def nama_kategori(self) -> str:
        return self.kategori.nama if self.kategori else "-"
```

Revisi 0003_tabel_kategori.py memuat tiga langkah: membuat tabel beserta data awal, menambah FK, lalu **memindahkan data lama** sebelum kolom teks dihapus:

```
KATEGORI_AWAL = ["Makan", "Transport", "Belajar", "Hiburan", "Lainnya"]

def upgrade() -> None:
    kategori = op.create_table(
        "kategori",
        sa.Column("id", sa.Integer(), primary_key=True),
        sa.Column("nama", sa.String(30), nullable=False, unique=True),
        sa.Column("urutan", sa.Integer(), nullable=False, server_default="0"),
        sa.Column("aktif", sa.Boolean(), nullable=False, server_default=sa.true()),
        sa.Column("created_at", sa.DateTime(), nullable=False, server_default=sa.func.now()),
        sa.Column("updated_at", sa.DateTime(), nullable=False, server_default=sa.func.now()),
    )
    # Data awal sama dengan daftar tetap di pertemuan 7.
    op.bulk_insert(
        kategori,
        [{"nama": nama, "urutan": i + 1} for i, nama in enumerate(KATEGORI_AWAL)],
    )

    op.add_column("kegiatan", sa.Column("kategori_id", sa.Integer(), nullable=True))
    op.create_index("ix_kegiatan_kategori_id", "kegiatan", ["kategori_id"])
    op.create_foreign_key(
        "fk_kegiatan_kategori", "kegiatan", "kategori",
        ["kategori_id"], ["id"], ondelete="RESTRICT",
    )

    # Migrasi data: teks kategori lama -> id kategori.
    op.execute()
```

```

"UPDATE kegiatan k JOIN kategori c ON c.nama = k.kategori "
"SET k.kategori_id = c.id WHERE k.tipe = 'pengeluaran'"
)
op.drop_column("kegiatan", "kategori")

```

Tulis juga `downgrade()` yang mengembalikan kolom teks dari hasil *join* (lihat snapshot). Uji keduanya:

```
docker compose exec api sh -c 'alembic downgrade -1 && alembic upgrade head && alembic check'
```

Bagian B: API Kategori

Di `schemas.py`, hapus `KATEGORI_PENGELUARAN`. Ganti `kategori: str` dengan `kategori_id: int | None` pada `KegiatanBase` dan `KegiatanPatch`. Validator kini hanya mewajibkan `kategori_id` untuk pengeluaran ("kategori wajib dipilih untuk pengeluaran") dan mengosongkannya untuk jenis lain. Respons tetap menyertakan **nama** kategori untuk ditampilkan:

```

class KegiatanOut(KegiatanBase):
    model_config = ConfigDict(from_attributes=True)

    id: int
    # Nama kategori untuk ditampilkan; dibaca dari properti model nama_kategori.
    kategori: str = Field(default="-", validation_alias="nama_kategori")
    ...

class KategoriOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: int
    nama: str

```

Keberadaan kategori dicek di router karena membutuhkan query. Kategori nonaktif ditolak untuk data baru, tetapi tetap boleh dipakai oleh kegiatan lama yang sudah memakainya:

```

def _cek_kategori(db: Session, data: KegiatanInput, lama: Kegiatan | None = None) -> None:
    if data.kategori_id is None:
        return
    kat = db.get(Kategori, data.kategori_id)
    tetap_sama = lama is not None and lama.kategori_id == data.kategori_id
    if kat is None or (not kat.aktif and not tetap_sama):
        raise RequestValidationError([
            {
                "type": "value_error",
                "loc": ("body", "kategori_id"),
                "msg": "kategori tidak ditemukan atau sudah nonaktif",
                "input": data.kategori_id,
            }
        ])

```

Panggil `_cek_kategori` di POST, PUT, dan PATCH. Pada GET `/kegiatan`, tambahkan `.options(joinedload(Kegiatan.kategori))` agar nama kategori ikut terambil dalam **satu** query. Tanpa ini, setiap baris memicu satu query tambahan (masalah **N+1 query**). Laporan per kategori kini memakai JOIN:

```

select (Kategori.nama.label("kategori"), func.sum(Kegiatan.nominal).label("total"))
.join(Kegiatan.kategori)
...
.group_by(Kategori.nama)

```

Endpoint baru `app/routers/kategori.py`:

```

@router.get("", response_model=list[KategoriOut])
def daftar_kategori(db: DbSession, _: UserAktif):
    """Kategori pengeluaran yang aktif,urut sesuai pengaturan admin."""
    stmt = (
        select(Kategori)
        .where(Kategori.aktif.is_(True))
        .order_by(Kategori.urutan, Kategori.nama)
    )

```

```
)  
return db.scalars(stmt).all()
```

Checkpoint B: GET /kategori membalas 5 kategori. Kegiatan pengeluaran lama kini memiliki kategori_id yang benar. POST pengeluaran tanpa kategori_id atau dengan id 99 dibalas 422.

Bagian C: Kategori Dinamis di Flutter

1. Model baru lib/models/kategori.dart (Kategori(id, nama) + fromJson) dan kategoriBawaan (5 kategori ber-id 1–5) untuk mode lokal SQLite.
2. Kegiatan mendapat field int? kategoriId.toJson() mengirim kategori_id (bukan nama), sedangkan fromJson() membaca kategori_id dan kategori (nama). SQLite tetap menyimpan nama; id dicari dari kategoriBawaan.
3. KategoriApi.daftar() memanggil GET /kategori lewat CacheClient(ApiClient()), sehingga kategori juga tersedia saat offline. Tambahkan Future<List<Kategori>> daftarKategori() ke kontrak repository.
4. Di form, chip kategori dibuat dari FutureBuilder<List<Kategori>>. Kategori lama yang sudah dinonaktifkan tetap ditampilkan agar kegiatan lama dapat disimpan ulang:

```
FutureBuilder<List<Kategori>>(  
  future: _daftarKategori, // = repo.daftarKategori() di initState  
  builder: (context, snapshot) {  
    if (snapshot.hasError) {  
      return TextButton.icon(  
        onPressed: _muatKategori,  
        icon: const Icon(Icons.refresh),  
        label: const Text('Gagal memuat kategori. Coba lagi'),  
      );  
    }  
    if (!snapshot.hasData) return const LinearProgressIndicator();  
    final daftar = [...snapshot.data!];  
    final dipilih = _kategori;  
    if (dipilih != null && !daftar.any((c) => c.id == dipilih.id)) {  
      daftar.add(dipilih);  
    }  
    return Wrap(  
      spacing: 8,  
      children: [  
        for (final c in daftar)  
          ChoiceChip(  
            label: Text(c.nama),  
            selected: _kategori?.id == c.id,  
            onSelected: (_) => setState(() {  
              _kategori = c;  
              _galatKategori = null;  
            }),  
          ),  
      ],  
    );  
  },  
)
```

Karena ChoiceChip bukan FormField, validasi "Pilih kategori pengeluaran" dilakukan di _simpan() dan pesannya ditampilkan dengan Text berwarna colorScheme.error.

5. Tambahkan widget test: chip berasal dari repository, pengeluaran tanpa kategori ditolak, dan kategori nonaktif milik kegiatan lama tetap tampil terpilih.

Checkpoint C: form menampilkan chip dari server. Kegiatan lama tetap menampilkan nama kategori yang benar. flutter test lulus.

5. Langkah Praktikum (Sesi 2: Dashboard)

Bagian D: Membuat Proyek Laravel + Filament

Semua perintah dijalankan di container PHP sementara, jadi tidak perlu memasang PHP. Buat image dasar sekali saja (simpan sebagai `Dockerfile.base` di folder kerja mana pun):

```
FROM dunglas/frankenphp:1-php8.4
RUN install-php-extensions pdo_mysql intl zip opcache
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
ENV COMPOSER_ALLOW_SUPERUSER=1
WORKDIR /app
```

```
docker build -t tabungku-php-base -f Dockerfile.base .
docker run --rm -v "$PWD":/app tabungku-php-base sh -c '
  composer create-project laravel/laravel admin "^13.0" &&
  cd admin &&
  composer require "filament/filament:^5.0" &&
  php artisan filament:install --panels --no-interaction'
```

Hasilnya adalah folder `admin/` berisi Laravel 13 + panel Filament 5 di `/admin`. Hapus folder `vendor/` dan berkas `database/database.sqlite` dari folder tersebut; `vendor` akan dipasang ulang di dalam image. Bila installer membuat berkas `AGENTS.md/CLAUDE.md`, hapus juga karena tidak dibutuhkan.

Bagian E: Login Admin di Tabel `admins`

1. Ubah nama `database/migrations/0001_01_01_000000_create_users_table.php` menjadi `..._create_admins_table.php`. Ganti `Schema::create('users', ...)` menjadi `'admins'` dan `password_reset_tokens` menjadi `admin_password_reset_tokens` (juga pada `down()`).
2. Hapus `app/Models/User.php` dan `database/factories/UserFactory.php`, lalu buat `app/Models/Admin.php`:

```
#[Fillable(['name', 'email', 'password'])]
#[Hidden(['password', 'remember_token'])]
class Admin extends Authenticatable implements FilamentUser
{
    protected function casts(): array
    {
        return ['email_verified_at' => 'datetime', 'password' => 'hashed'];
    }

    /** Wajib di APP_ENV=production: siapa yang boleh membuka panel. */
    public function canAccessPanel(Panel $panel): bool
    {
        return true;
    }
}
```

3. Di `config/auth.php`, ubah model provider menjadi `Admin::class` dan tabel *password reset* menjadi `admin_password_reset_tokens`.
4. `database/seeder/DatabaseSeeder.php` membuat admin pertama dari *environment*. Aman dijalankan berulang:

```
Admin::firstOrCreate(
    ['email' => env('ADMIN_EMAIL', 'admin@tabungku.test')],
    ['name' => 'Administrator', 'password' => env('ADMIN_PASSWORD', 'admin12345')],
);
```

5. `routes/web.php`: `Route::redirect('/', '/admin');`. Di `config/app.php`: `'timezone' => env('APP_TIMEZONE', 'Asia/Jakarta')`.

Bagian F: Menambahkan Layanan admin ke Compose

admin/Dockerfile memakai FrankenPHP (PHP + web server dalam satu image):

```
FROM dunglas/frankenphp:1-php8.4

RUN install-php-extensions pdo_mysql intl zip opcache
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
ENV COMPOSER_ALLOW_SUPERUSER=1

WORKDIR /app

# Dependensi dipasang lebih dulu agar layer ini di-cache.
COPY composer.json composer.lock ./
RUN composer install --no-interaction --prefer-dist --no-scripts --no-autoloader

COPY . .
RUN composer dump-autoload --optimize

EXPOSE 8080
CMD ["sh", "docker/start.sh"]
```

admin/docker/start.sh menunggu tabel milik Alembic, menjalankan migrasi milik Laravel, membuat admin, lalu menyalakan server:

```
#!/bin/sh
set -e

if [ ! -f vendor/autoload.php ]; then
    composer install --no-interaction --prefer-dist
fi
[ -f .env ] || touch .env
mkdir -p storage/framework/cache storage/framework/sessions \
    storage/framework/views storage/logs bootstrap/cache

echo "Menunggu tabel milik FastAPI (Alembic)..."
until php artisan db:table kategori >/dev/null 2>&1; do sleep 2; done

php artisan migrate --force # hanya tabel milik Laravel (admins, sessions, cache, jobs)
php artisan db:seed --force # akun admin pertama
php artisan filament:upgrade # salin aset CSS/JS Filament ke public/

exec frankenphp php-server --root public/ --listen :8080
```

Tambahkan juga admin/.dockerignore (vendor, node_modules, .env, isi storage/, aset public/*/filament). Di docker-compose.yml, beri layanan api sebuah *healthcheck* (python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"), lalu tambahkan layanan baru:

```
admin:
  build: ./admin
  restart: unless-stopped
  environment:
    APP_NAME: "TabungKu Admin"
    APP_ENV: local
    APP_DEBUG: "true"
    APP_KEY: ${ADMIN_APP_KEY}
    APP_URL: http://localhost:8080
    APP_LOCALE: id
    LOG_CHANNEL: stderr
    DB_CONNECTION: mysql
    DB_HOST: mysql
    DB_PORT: 3306
    DB_DATABASE: ${MYSQL_DATABASE}
    DB_USERNAME: ${MYSQL_USER}
    DB_PASSWORD: ${MYSQL_PASSWORD}
    SESSION_DRIVER: database
    CACHE_STORE: database
    QUEUE_CONNECTION: sync
```

```

ADMIN_EMAIL: ${ADMIN_EMAIL}
ADMIN_PASSWORD: ${ADMIN_PASSWORD}
TZ: Asia/Jakarta
ports:
  - "8080:8080"
volumes:
  - ./admin:/app # kode Laravel di laptop langsung dipakai container
  - /app/vendor # vendor tetap dari image (tidak tertimpa folder laptop)
healthcheck:
  test: ["CMD", "curl", "-fs", "http://localhost:8080/up"]
  interval: 10s
  timeout: 5s
  retries: 10
  start_period: 30s
depends_on:
  mysql:
    condition: service_healthy
  api:
    condition: service_healthy

```

Tambahkan ke `.env.example` `ADMIN_APP_KEY=base64:...` (buat dengan `docker compose run --rm admin php artisan key:generate --show`), `ADMIN_EMAIL`, dan `ADMIN_PASSWORD`. Jalankan `docker compose up -d --build`, lalu buka `http://localhost:8080/admin` dan login.

Checkpoint F: login dashboard berhasil. Di Adminer ada tabel `admins`, `sessions`, dan `migrations` di samping tabel milik Alembic, dan tabel `users` tidak berubah.

Bagian G: Model Eloquent untuk Tabel Milik Alembic

Laravel 13 dapat menyatakan nama tabel dengan *attribute* `#[Table]. app/Models/Pengguna.php`:

```

/**
 * Pengguna aplikasi mobile. Tabel `users` dibuat dan dimigrasi oleh
 * Alembic (FastAPI), jadi TIDAK ada migrasi Laravel untuk tabel ini.
 */
#[Table('users')]
#[Fillable(['nama', 'target_harian', 'aktif'])]
#[Hidden(['password_hash'])]
class Pengguna extends Model
{
    // Tabel users hanya punya created_at (tanpa updated_at).
    const UPDATED_AT = null;

    protected function casts(): array
    {
        return ['aktif' => 'boolean', 'target_harian' => 'integer'];
    }

    public function kegiatan(): HasMany
    {
        return $this->hasMany(Kegiatan::class, 'user_id');
    }
}

```

Buat juga `Kategori` (`#[Table('kategori')]`), *fillable* nama, urutan, aktif, relasi `hasMany` ke `Kegiatan` dan `Kegiatan` (`#[Table('kegiatan')]`), *cast* tanggal sebagai `date`, relasi `belongsTo` ke `Pengguna` lewat `user_id` dan ke `Kategori`).

Bagian H: Filament Resource

Buat kerangka dengan generator. Opsi `--simple` menghasilkan satu halaman dengan form dalam *modal*, dan `--generate` membaca kolom dari database:

```

docker compose exec admin php artisan make:filament-resource Kategori \
--simple --generate --record-title-attribute=nama
docker compose exec admin php artisan make:filament-resource Pengguna \
--simple --generate --record-title-attribute=nama

```

```
docker compose exec admin php artisan make:filament-resource Kegiatan \
--simple --generate --record-title-attribute=judul
```

Generator memakai aturan jamak bahasa Inggris, sehingga foldernya bernama **Kategoris**, **Penggunas**, dan **Kegiatans**. Label tampilan diperbaiki dengan `$modelLabel` dan `$pluralModelLabel`.

KategoriResource (CRUD lengkap, urutan dapat diubah dengan *drag-and-drop*):

```
public static function form(Schema $schema): Schema
{
    return $schema->components([
        TextInput::make('nama')->required()->maxLength(30)->unique(ignoreRecord: true),
        TextInput::make('urutan')->numeric()->minValue(0)->default(0)
            ->helperText('Urutan tampil di aplikasi (kecil di depan)'),
        Toggle::make('aktif')->default(true)
            ->helperText('Kategori nonaktif tidak muncul di aplikasi.'),
    ]);
}

public static function table(Table $table): Table
{
    return $table
        ->defaultSort('urutan')
        ->reorderable('urutan')
        ->columns([
            TextColumn::make('nama')->searchable(),
            TextColumn::make('urutan')->sortable(),
            ToggleColumn::make('aktif'),
            TextColumn::make('kegiatan_count')->counts('kegiatan')
                ->label('Dipakai')->suffix(' kegiatan'),
        ])
        ->recordActions([
            EditAction::make(),
            // Kategori yang sudah dipakai tidak boleh dihapus (FK RESTRICT).
            DeleteAction::make()
                ->hidden(fn (Kategori $record): bool => $record->kegiatan()->exists()),
        ]);
}
```

PenggunaResource: `canCreate()` mengembalikan `false`. Form hanya berisi nama, email (`disabled()->dehydrated(false)`), `target_harian`, dan `aktif`. Tabel menampilkan nama, email, `target` (`->money('IDR', locale: 'id', decimalPlaces: 0)`), jumlah kegiatan (`counts('kegiatan')`), `ToggleColumn::make('aktif')`, tanggal daftar, dan `TernaryFilter::make('aktif')`. Hapus `CreateAction` dari `ManagePenggunas`.

KegiatanResource: baca-saja (`canCreate()` `false`, tanpa form), dengan `ViewAction` (*infolist*) dan `DeleteAction`:

```
->columns([
    TextColumn::make('tanggal')->date('d M Y')->sortable(),
    TextColumn::make('judul')->searchable(),
    TextColumn::make('pengguna.nama')->label('Pengguna')->searchable(),
    TextColumn::make('tipe')->badge()
        ->formatStateUsing(fn (string $state): string => self::TIPE[$state])
        ->color(fn (string $state): string => match ($state) {
            'pemasukan' => 'success',
            'pengeluaran' => 'danger',
            default => 'gray',
        }),
    TextColumn::make('nominal')
        ->money('IDR', locale: 'id', decimalPlaces: 0)->sortable(),
    TextColumn::make('kategori.nama')->label('Kategori')->placeholder('-'),
    IconColumn::make('selesai')->boolean(),
])
->filters([SelectFilter::make('tipe')->options(self::TIPE)])
```


Terakhir, di `AdminPanelProvider`, ganti warna menjadi `Color::Teal`, tambahkan `->brandName('TabungKu Admin')`, dan hapus `FilamentInfoWidget`.

Bagian I: Uji Alur Ujung ke Ujung

No	Langkah di dashboard	Yang diamati di aplikasi / API
1	Tambah kategori "Kos" urutan 6	Form kegiatan di HP menampilkan chip Kos (buka ulang form)
2	Drag "Belajar" ke posisi pertama	Urutan chip di HP berubah
3	Nonaktifkan "Hiburan"	Chip hilang di form baru; kegiatan lama ber-kategori Hiburan tetap dapat diubah
4	Coba hapus "Makan" yang sudah dipakai	Tombol hapus tidak tersedia
5	Matikan saklar aktif pengguna Budi	Login Budi di HP: "Akun dinonaktifkan oleh admin"; request yang sudah login dibalas 403
6	Tambah kegiatan di HP	Muncul di tabel Kegiatan dashboard (muat ulang), lengkap dengan nama pengguna

Untuk mengisi data contoh agar tabel tidak kosong: `docker compose exec api python -m scripts.seed_demo` (3 akun, kegiatan 30 hari, password `rahasia123`).

Smoke test dashboard dapat dijalankan dengan `docker compose exec admin php artisan test` (lihat `tests/Feature/DashboardTest.php` di snapshot). Test ini memakai database docker compose karena tabel domain dibuat oleh Alembic.

Checkpoint I: keenam skenario berhasil dan didokumentasikan dengan tangkapan layar.

6. Latihan Mandiri

- Tambahkan kolom pencarian email pada filter Pengguna dan tampilkan total nominal pengeluaran setiap pengguna (`->sum('kegiatan', 'nominal')`).
- Tambahkan `BulkAction` untuk menonaktifkan banyak kategori sekaligus.
- Publikasikan berkas bahasa Laravel (`php artisan lang:publish`), lalu terjemahkan pesan validasi `required` dan `unique` ke Bahasa Indonesia.
- Jelaskan mengapa `ToggleColumn` pada pengguna lebih tepat daripada tombol **Hapus** pengguna.

7. Tugas

Lengkapi dashboard agar berguna bagi pengelola:

- Widget statistik** (`StatsOverviewWidget`) untuk hari ini: jumlah pengguna aktif (beserta jumlah pendaftar hari ini), jumlah kegiatan (beserta jumlah yang selesai, dengan *sparkline* 7 hari), total pemasukan, dan total pengeluaran semua pengguna dalam format Rupiah.
- Grafik donat** pengeluaran per kategori, dengan pilihan rentang 7 hari / 30 hari / 1 tahun (`getFilters()`).
- Grafik garis** arus kas 14 hari terakhir (pemasukan vs pengeluaran). Hari tanpa data tetap tampil dengan nilai 0.
- Filter tabel Kegiatan: kategori (`relationship`), pengguna (dapat dicari), dan **rentang tanggal** (dua `DatePicker` dengan indikator filter aktif).
- Feature test Livewire untuk widget dan filter (`filterTable`, `assertCanSeeTableRecords`).

- Kumpulkan tangkapan layar dashboard yang berisi data (gunakan `seed_demo`), serta video atau GIF singkat skenario 1 dan 5 pada bagian I.

8. Rubrik Penilaian

Komponen	Bobot
Tabel kategori, migrasi data, API kategori, kategori dinamis di Flutter (A–C)	25%
Layanan admin di Compose, login di tabel <code>admins</code> , model Eloquent (D–G)	20%
Tiga Resource sesuai ketentuan (H)	15%
Uji ujung ke ujung (I)	10%
Tugas: widget statistik dan dua grafik	20%
Tugas: filter tabel dan test	10%

9. Pertanyaan Refleksi

1. Apa yang akan terjadi bila Laravel juga memiliki migrasi untuk tabel kegiatan?
2. Mengapa hapus kategori diganti dengan menonaktifkan? Apa peran `ON DELETE RESTRICT`?
3. Jelaskan masalah N+1 query dan bagaimana `joinedload` (SQLAlchemy) maupun relasi Eloquent mengatasinya.
4. Admin menonaktifkan akun, tetapi pengguna masih memegang access token yang berlaku 15 menit. Apakah pengguna itu masih bisa mengakses data? Tunjukkan baris kode yang menjawabnya.
5. Bandingkan usaha membuat halaman CRUD kategori di Filament dengan membuat layar CRUD di Flutter.

10. Troubleshooting Umum

Masalah	Solusi
<code>SQLSTATE[42S01] Table 'users' already exists saat migrate</code>	Migrasi bawaan belum diubah. Ganti ke tabel <code>admins</code> (bagian E)
Dashboard tampil tanpa CSS	Aset belum dipublikasikan: <code>docker compose exec admin php artisan filament:upgrade</code>
<code>Class "App\Models\User" not found</code>	Masih ada rujukan ke User (misalnya <code>config/auth.php</code> atau seeder). Ganti ke Admin
Container admin terus mengulang "Menunggu tabel milik FastAPI"	Migrasi Alembic gagal. Periksa <code>docker compose logs api</code>
Paket baru di <code>composer.json</code> tidak terbaca	Volume anonim <code>/app/vendor</code> masih berisi vendor lama: <code>docker compose up -d --build --renew-anon-volumes admin</code>
<code>MissingAppKeyException</code>	Isi <code>ADMIN_APP_KEY</code> di <code>.env</code> (bagian F)
Pesan validasi berbahasa Inggris	Terjemahan Filament sudah ada (<code>APP_LOCALE=id</code>), sedangkan pesan validasi Laravel perlu dipublikasikan dan diterjemahkan (latihan 3)
Migrasi 0003 gagal pada <code>UPDATE ... JOIN</code>	Sintaks tersebut khusus MySQL. Pastikan memakai MySQL, bukan SQLite

11. Referensi

- Filament 5, Resources dan Tables: filamentphp.com/docs
- Laravel 13, Eloquent: laravel.com/docs/eloquent
- FrankenPHP: frankenphp.dev
- SQLAlchemy, *Relationship Loading Techniques*:
docs.sqlalchemy.org/en/20/orm/queryguide/relationships.html